

Unidad 6



Sistemas domóticos basados en electrónica. Arduino

Instalaciones domóticas



Índice



6.1. Los sistemas domóticos basados en electrónica

6.2. El sistema de desarrollo basado en Arduino

6.3. Tipos de placas Arduino

6.4. El lenguaje de programación

6.5. El entorno de programación

6.6. La programación de la placa Arduino

6.6.1. Configuración de hardware

6.6.2. La estructura de un programa de Arduino

6.6.3. La compilación y transferencia

6.7. Las entradas y salidas digitales

6.7.1. Las salidas digitales

6.7.2. Las entradas digitales

6.7.3. El monitor serie

6.8. Las entradas analógicas

6.9. La conexión de cargas

6.9.1. El relé

6.9.2. El transistor

6.10. Actuadores

6.10.1. El diodo led y la regulación PWM

6.10.2. El diodo led RGB

6.10.3. Pantalla LCD y el bus I2C

6.10.4. El zumbador

6.11. Sensores

6.11.1. El sensor de temperatura y humedad

6.11.2. El sensor de ultrasonidos

6.12. Servidor domótico



Introducción

Los sistemas domóticos se basarán en la electrónica y eléctrica, en este caso hablaremos de la placa Arduino ya que es una herramienta muy eficiente y rentable económicamente, además de que su lenguaje de programación es muy fácil de interpretar.

Al finalizar esta unidad

- + Distinguiremos los diferentes modelos de Arduino
- + Aprenderemos a configurar placas
- + Realizaremos configuraciones mediante monitores serie

6.1.

Los sistemas domóticos basados en electrónica

Este tipo de sistemas fueron el punto de partida en la domótica. Estos empezaron como sistemas cerrados en empresas que desarrollaban el funcionamiento interno. El problema es que no estaba estandarizado, por lo que cualquier tipo de cambio o añadido a nuestro programa teníamos que decírselo a la empresa para que lo realizase.

Hoy en día, ha cambiado completamente puesto que ya está estandarizado y cualquier persona en su casa podría hacer un sistema domotizado, ya sea más o menos sofisticado.

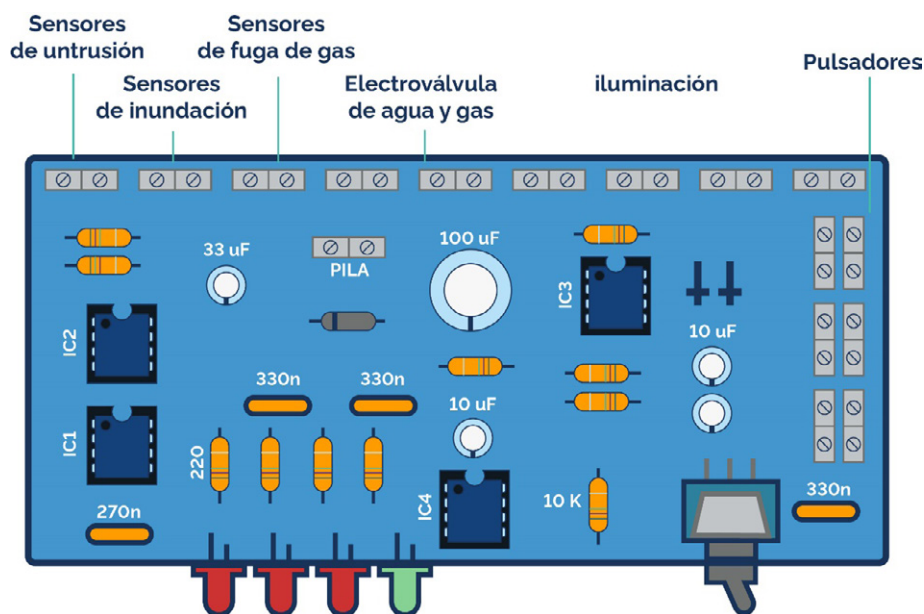


Imagen 1. Sistema electrónico de domótica

6.2.

El sistema de desarrollo basado en Arduino

Arduino se trata de un código abierto que realiza el desarrollo de proyectos de electrónica. Al principio se creó para aprender a programar electrónica básica y poco a poco ha ido mejorando hasta tal punto de que existen viviendas que se han podido domotizar con el sistema Arduino.

6.3.

Tipos de placas Arduino

Son microcontroladores que tienen un sistema hardware abierto, lo que quiere decir que cualquier persona puede utilizar estos esquemas y desarrollar su propio código. Hay varios tipos de placas de Arduino como son:

- > **Arduino Uno.** Es el más usado. Cuenta con 15 pines digitales y 5 analógicos. El procesador será el ATMEGA328P de 32 kB de memoria.
- > **Arduino Leonardo.** Es muy parecido al Arduino Uno. Tendrá un procesador ATMEGA32U4 y dispondrá de 20 pines de entrada y salidas digitales.
- > **Arduino 101.** Tiene un microprocesador Intel Curie, por lo que se considera una placa de alto rendimiento con bajo consumo.
- > **Arduino Explora.** Es una evolución de la placa Leonardo que integrara micrófono, avisador acústico, sensor de temperatura, acelerómetro, led RGB, cuatro pulsadores y un joystick.
- > **Arduino Zero.** Será una evolución de Arduino Uno, pero con más potencia y un microprocesador SAMD21. Esta placa trabajara con 3,3V mientras que el resto la mayoría con 5V.
- > **Arduino Mega.** Cuando el Arduino Uno se nos quede un poco corto con las prestaciones necesarias, usaremos esta placa. Con un microprocesador ATMEGA256. Con 70 pines analógicos y digitales y 256 KB de memoria flash.
- > **Arduino Mini Pro.** Tiene un microprocesador ATMEGA328P con dimensiones reducidas. Cuenta con 14 pines de entrada y salida. Con dos modelos de 3,3 V y otro de 5V

Estas placas Arduino tendrán dos puertos: Las **entradas** que son por donde reciben las señales que provienen de pulsadores y sensores y las **salidas** que es por donde escribe o envía señales para activar el receptor con ellos.



Imagen 2. Placas Arduino

6.4.

El lenguaje de programación

Un programa informático tendrá un conjunto de instrucciones que serán las encargadas de decirle al microprocesador como deberá actuar. Arduino se programará con una variante del lenguaje C++.

El microprocesador no entenderá este lenguaje tal cual por lo que siempre tendremos que compilar el programa para que lo pase a código máquina y así pueda entenderlo.

6.5.

El entorno de programación

El entorno de programación, también llamado IDE (integrated development environment), será donde realizaremos el programa o código fuente.

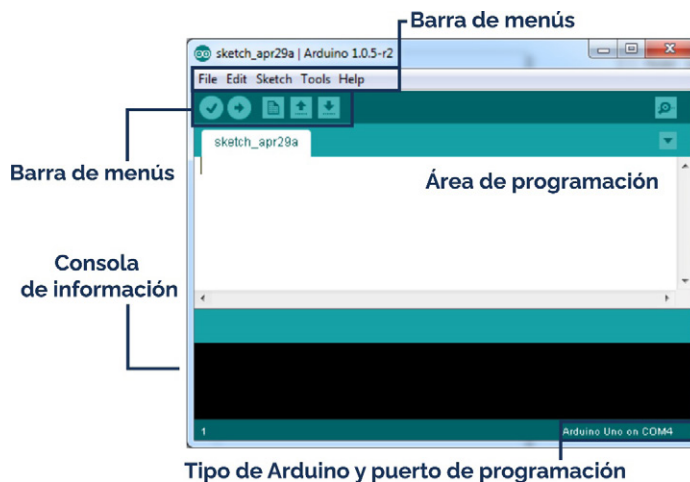


Imagen 3. Partes más importantes de un programa

Cuando hayamos instalado el programa en www.arduino.cc. Una vez instalado mostrará el IDE, el cual tiene las siguientes partes:

- > **Barra de menús:** contiene varias opciones que puede realizar el IDE
- > **Barra de iconos:** es la barra de accesos que más usamos.
- > **Área de programación:** espacio para poder programar
- > **Consola de información:** se muestra información del proceso de compilación y transferencia del programa
- > **Tipo de Arduino y puerto de programación:** Informa sobre el tipo de placa que se estará realizando la programación.



6.6.

La programación de la placa Arduino

La programación de una placa Arduino comenzará en el hardware y seguidamente se programará el código que queremos emplear y lo transferiremos a una placa Arduino por medio de un USB.

6.6.1. Configuración de hardware

En el entorno IDE podremos usar diferentes tipos de placas para programar. Por ello en el menú de herramientas, seleccionaremos la placa que queremos usar. También tendremos que usar un puerto USB, normalmente aparece como COM, para realizar la comunicación.

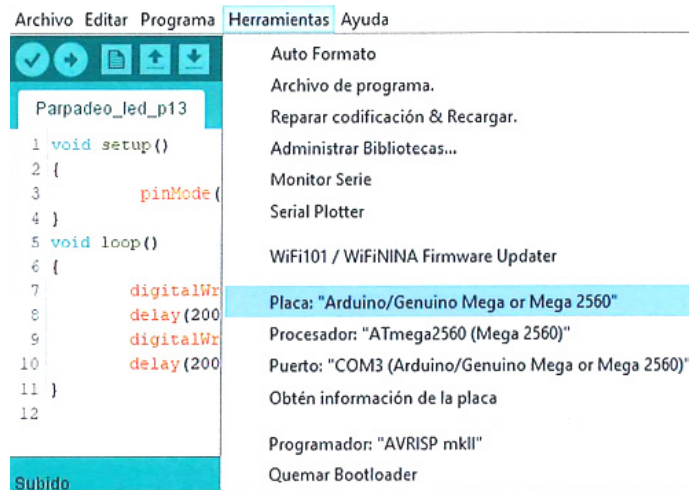


Imagen 4. Configuración de la placa

6.6.2. La estructura de un programa de Arduino

Un programa de Arduino consistirá en dos funciones básicas:

1. **Setup.** En esta función, indicaremos las instrucciones solo cuando iniciemos el programa, es decir, al arrancarlo.
2. **Loop.** Es una función que se repite constantemente.

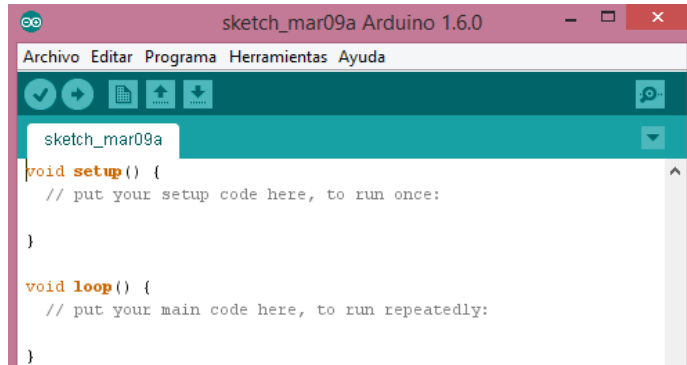


Imagen 5. Funciones loop y setup

6.6.3. La compilación y transferencia

Cuando ya hayamos hecho el programa, lo siguiente que tendremos que comprobar es si el programa no tiene fallos. En el IDE de Arduino, se podrá compilar desde el menú programa y luego clicando sobre el icono de verificar/compilar.

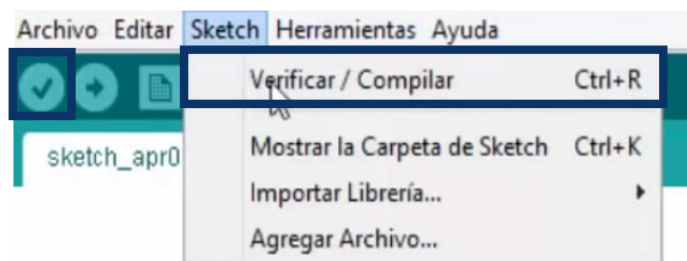


Imagen 6. Compilación



6.7.

Las entradas y salidas digitales

Estas entradas y salidas usarán valores binarios (1 o 0) y pondrán a la salida los mismos valores.

6.7.1. Las salidas digitales

Las salidas digitales permitirán poner un valor binario de 0 o 1 a la salida. Algunos de los pines los podremos configurar de diferentes maneras dependiendo de para que los usemos. Para configurar este puerto será mediante la instrucción **pinMode**. Esta instrucción necesitara dos valores (el numero de puerto y como debe trabajar). Si quisiéramos poner un puerto como salida se hará de la siguiente manera:

```
pinMode (13, OUTPUT);
```

Habrà que prestar especial atención a las mayúsculas y al (;) del final. Este tipo de instrucciones solo la usaremos una vez, cuando arranca el programa, por lo que la pondremos en la función setup.

Para poder escribir un valor en el pin de salida usaremos **digitalWrite**, como en el ejemplo que pondremos a continuación con el puerto 13 y su valor binario;

```
digitalWrite (13, 1);
```

Este compilador por lo tanto entenderà que, si ponemos el paràmetro HIGH, se pondrà a 1 y si ponemos, LOW, se pondrà a 0.

```
digitalWrite (13, HIGH);
```

6.7.2. Las entradas digitales

Las entradas digitales como ya sabemos podrán tener dos valores (1 o 0). Cuando tenemos un pulsador que oscila entre 5V y 0V corresponderán a los valores 1 o 0. No tendremos que dejar el pin al aire, por lo que se empleará una resistencia de pullup (1) o pulldown. (0).

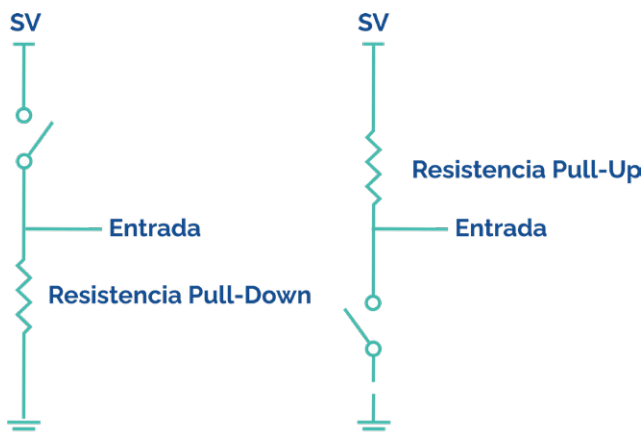


Imagen 7. Resistencia pullup y pulldown



Cuando leemos un valor de una de las entradas, se guardará en algún sitio de la memoria y la llamaremos variable. Cuando definimos estas variables tendremos que indicar al compilador cuanto debemos reservar y declararla antes de usarla.

Una variable solo va a guardar un bit (un 1 o 0) y lo declararemos como un **bool**. Cuando guardamos una variable como **int** será correspondiente a un valor entero (-32768 y 32767) y se declararán de la siguiente manera:

```
bool nombre_variable_1;
int nombre_variable_2;
```

Los nombres de las variables no podremos repetirlas, y podemos inicializarlas cuando las creamos y reservar ese espacio de memoria como, por ejemplo:

```
bool nombre_variable_1= false;
int nombre_variable_2= 123;
```

La lectura del valor digital la haremos mediante **digitalRead** y entre paréntesis pondremos el número de pin que queremos leer como, por ejemplo:

```
int valor;
valor = digitalRead (6);
```

6.7.3. El monitor serie

Podremos comunicar el Arduino por el puerto serie, entre el ordenador y este, y tendremos para ello una librería que se llama Serial. Dicha librería tendremos que inicializarla con una velocidad entre los 300 y 115200 bits por segundo, pero suele ser a 9600 bits por segundo.

Para enviar un dato, emplearemos la orden **println**, junto con el dato.

```
void setup ()
{
  Serial.begin (9600);
}
void loop ()
{
  Int i = 1234 ;
  Serial.println ( i );
}
```

Para leer los datos que envía Arduino por comunicación, debemos abrir el monitor serie en herramientas y ajustarlo a la velocidad que quiera ir.

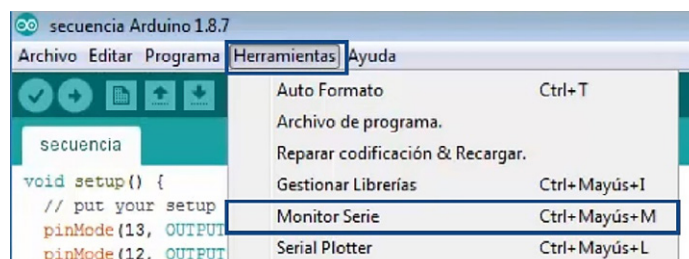


Imagen 8. Apertura del monitor serie

6.8.

Las entradas analógicas

Arduino tiene puertos analógicos, 6 en el modelo Uno y 16 en el Mega. Estos estarán etiquetados como A0-A15 con rotulación en la placa como ANALOG IN. Los puertos tienen un convertidor de analógico a digital (ADC), que tendrá un valor entre 5V y 0V.

Para poder saber cómo funcionan las entradas, colocaremos un potenciómetro en sus extremos y conectaremos el cursor a uno de los puertos analógicos, así que su tensión oscilara entre 5V y 0V.

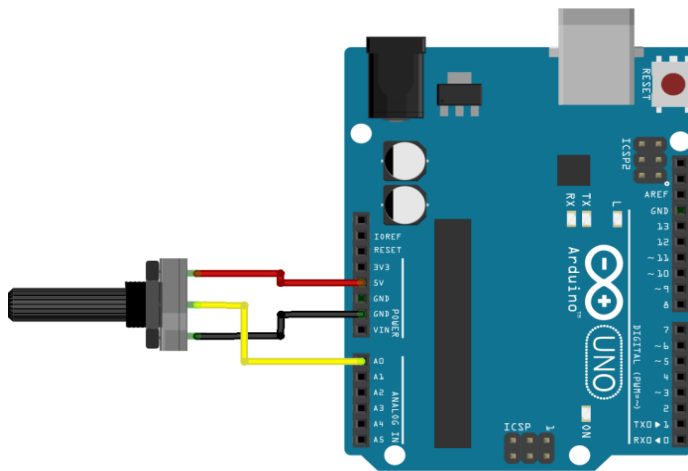


Imagen 9. Montaje de potenciómetro conectado a una entrada analógica

Estos convertidores de Arduino Uno y Mega serán de 10 bits de resolución, por lo que devolverá valores que estén entre 0 y 2^{10} = 1024 para tensiones que estén entre 0V y 5V.

La instrucción con la que leeremos por un puerto analógico será `analogRead`, seguido del número del puerto, por ejemplo:

```
int leer;  
leer = analogRead (A5);
```

6.9.

La conexión de cargas

Las salidas del Arduino trabajarán con corrientes muy pequeñas. Una solución a este problema será poner a algún elemento que trabaje como preactuador, para que pueda trabajar con corrientes más elevadas. Que serán básicamente el relé y el transistor.

6.9.1. El relé

El relé trabajará con la tensión de mando que será de 5V y para sus contactos tensiones que pueden alcanzar los 250 V con corrientes entre 5A y 10A. Tendrán tres bornes en la parte del mando (positivo, negativo y señal) y tendrá otros tres en la parte de la salida (común, normalmente abierto y cerrado).



Imagen 10. Relé para Arduino.
Fuente: todoelectronica.com

6.9.2. El transistor

El transistor lo usaremos cuando queramos controlar cargas. Es mas pequeño que el relé y velocidades de conmutación más rápidas. Hay dos tipos de transistores bipolares, el PNP y NPN, y en cada una de ellas la carga varía.

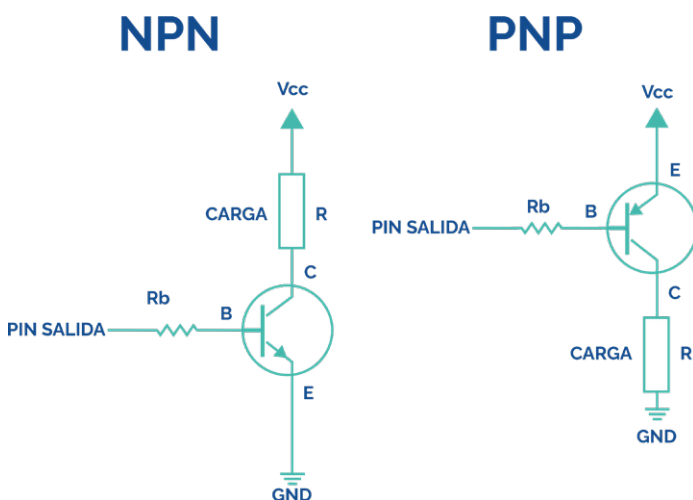


Imagen 11. Conexión de la carga según el tipo de transistor bipolar

Cuando tengamos una carga inductiva, como por ejemplo un motor, tendremos que colocar un diodo en paralelo con la carga. El diodo recibirá el nombre de flyback.

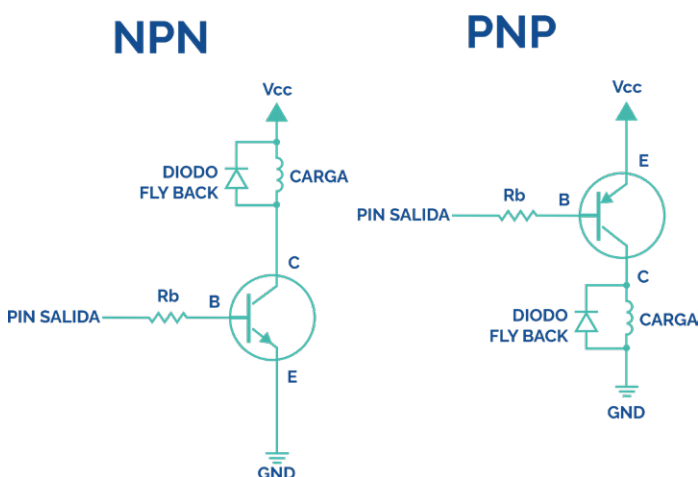


Imagen 12. Conexión del diodo en paralelo con la carga por proteger.

El valor de R_b dependerá de la V_{cc} , por ejemplo, para una tensión de 5V, el valor será de 220 o 330 Ω . Los transistores suelen ser baratos y hay una amplia gama, como el 2N2222 o BC546.



6.10.

Actuadores

Hay muchos actuadores que podremos conectar a la salida, los mas destacados los comentaremos a continuación.

6.10.1. El diodo led y la regulación PWM

Hemos estado estudiando cómo podemos activar o desactivar un diodo led, pero ahora veremos el nivel de iluminación que podemos ponerle. Prácticamente todos los Arduino no tienen salidas analógicas, por lo que se empleará la técnica PWM (pulse width modulation) que generará una onda cuadrada e irá variando su ancho en cada ciclo, así pues, el valor eficaz de la salida será igual a una salida analógica.

Si por ejemplo tenemos un 50% de ciclo trabajo, a la salida tendremos una señal analógica de 2,5 V. El valor se encontrará entre 0 y 255. Para usar la técnica de PWM, tendremos que ponerlo en los pines adecuados con la etiqueta PWM o el símbolo ~. La orden que debemos usar para trabajar con esta técnica será:

```
analogWrite (puerto, valor);
```

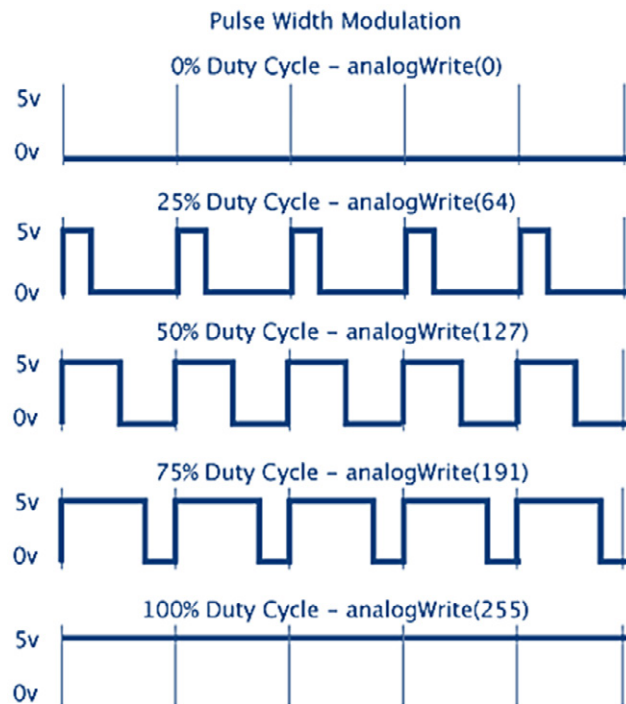


Imagen 13. Señal PWM

6.10.2. El diodo led RGB

Dependiendo del diodo y de sus características, emitirá un color u otro. Para conseguir que el diodo pueda emitir cualquier color se hará una mezcla con los colores primarios (rojo, verde y azul).

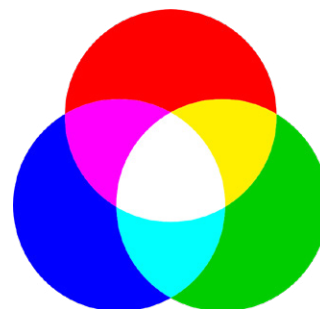


Imagen 14. Mezcla de colores primarios

6.10.3. Pantalla LCD y el bus I2C

El bus I2C se trata de una norma de comunicación entre componentes electrónicos que simplificará el cableado, ya que necesitará dos cables para el control (SDA y SCL) y los dos de alimentación. A este bus se le conectarán todos los buses en paralelo. Se compondrán de un maestro y el resto serán los esclavos, además de llevar dos resistencias que irán con el propio componente.

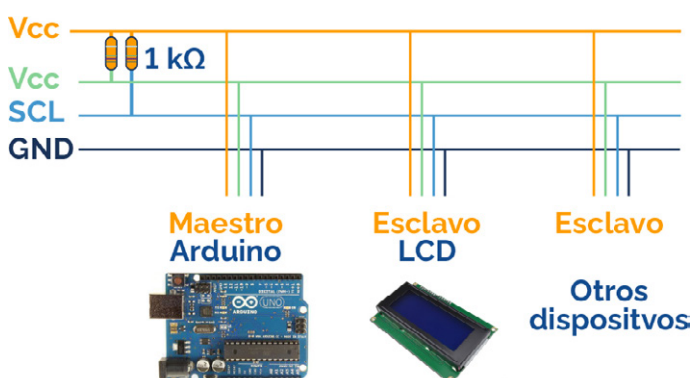


Imagen 15. El bus I2C

Dependiendo de la placa de Arduino, tendremos los siguientes pines:

- > En el Arduino Uno, los pines están en los analógicos A4 (SDA) y A5 (SCL).
- > En el Arduino Mega, serán el 20 (SDA) y 21 (SCL).

Tendremos algunos componentes electrónicos que necesitarán librerías, y serán el conjunto de ficheros con un código que le dicen a Arduino como tiene que trabajar. Cuando usemos la pantalla LCD, necesitaremos la librería NewLiquidCrystal_1.3.4.zip.

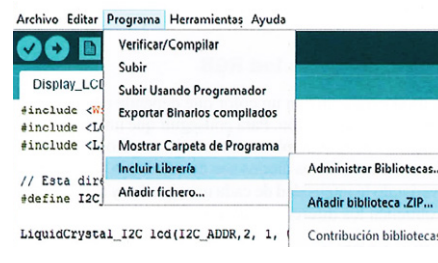


Imagen 16. Añadir biblioteca .ZIP.

6.10.4. El zumbador

Sera el encargado de dar avisos acústicos. Su efecto es causado por la piezoelectricidad, en la cual, cuando la sometemos a una corriente, el elemento se pondrá a vibrar.



Imagen 17. Zumbador

6.11.

Sensores

Existen cualquier tipo de sensor para medir cualquier variable física. El funcionamiento de la lectura de la variable de un sensor es muy parecido para todos ellos.

6.11.1. El sensor de temperatura y humedad

Existirán muchos tipos de sensores que nos den la lectura de la magnitud física y humedad. Haremos ahora una tabla comparativa entre los dos mas famosos DHT11 y DHT22.

	Comparativa entre DHT11 y DHT22	
	DHT11	DHT22
Humedad	Del 20% al 80% HR	Del 0% al 100% HR
Precisión de la humedad	±5 % HR	±2 % HR
Temperatura	De los 0°C a los 50°C	De los -40°C a los 80°C
Precisión de la temperatura	±2 °C	±0,5 °C
Numero de lecturas	1 por segundo	2 por segundo

Tendrán tanto la misma forma de conectarlos como la programación. Son capaces de hacer la realización de analógico a digital y nos darán el resultado por un pin digital.

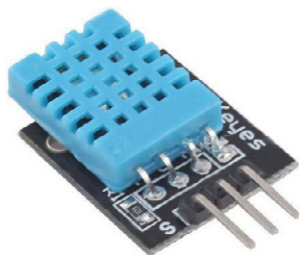


Imagen 18. Sensor DHT11



Imagen 19. Sensor DHT22

6.11.2. El sensor de ultrasonidos

Este sensor se usará para detectar objetos y así saber la distancia que hay entre este y el sensor. Consiste en enviar una onda de ultrasonidos para hacerla rebotar en el objeto y volver al sensor. Una vez recibe esa onda, dependiendo del tiempo que ha tardado podrá calcular la distancia a la que se encuentra del objeto.

Entre los modelos más importantes está el SRF05, que dispondrá de un emisor y receptor que funcionan a 5V. La velocidad será la de la luz 340 m/s.

La conexión serán dos pines para la alimentación, uno para el trigger o pulso de inicio y otro para el eco.

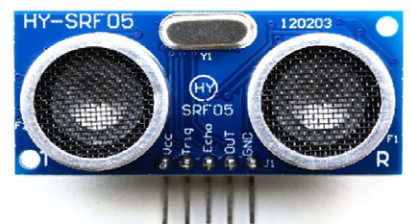


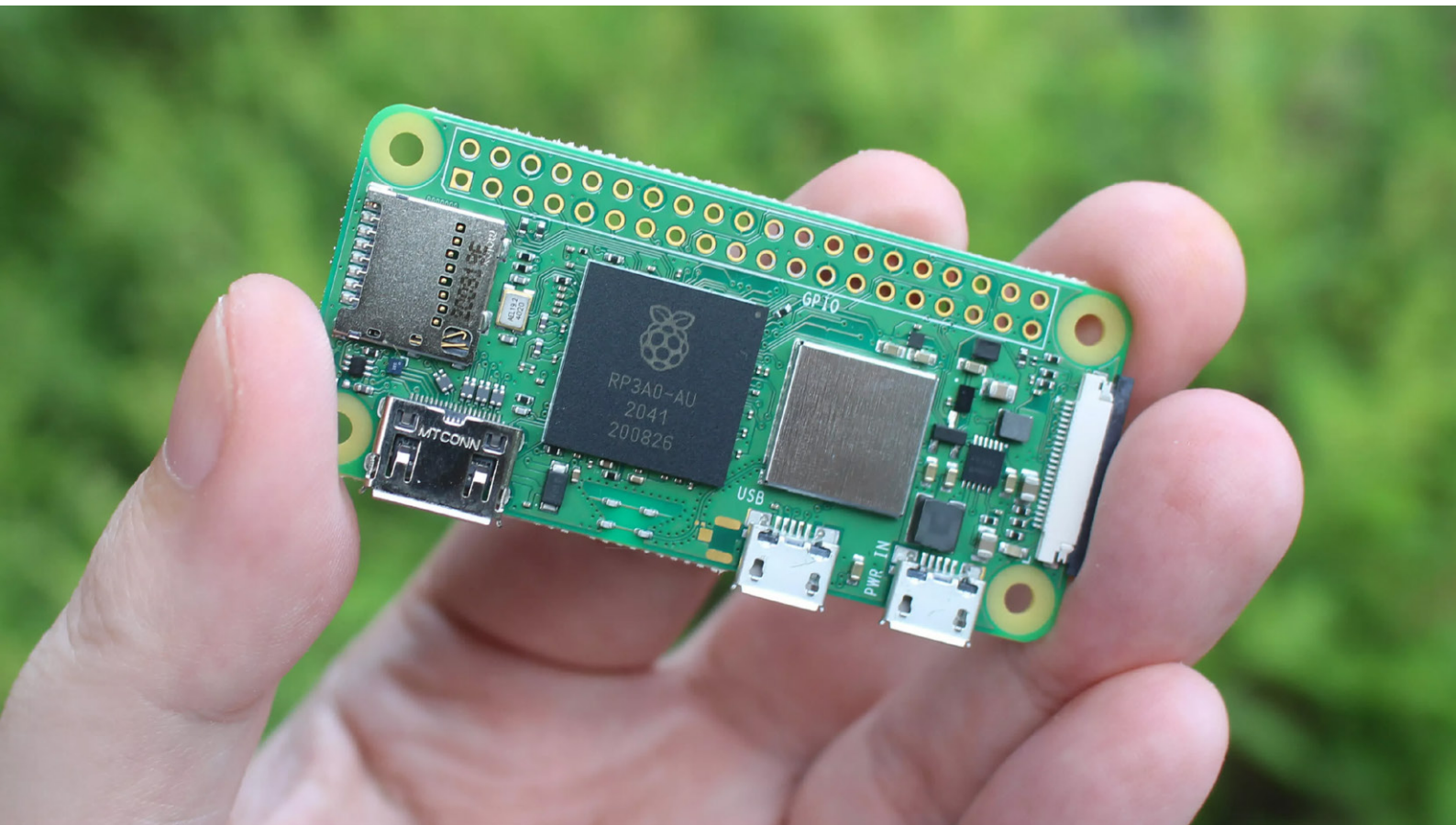
Imagen 20. Sensor de ultrasonidos SRF05

6.12.

Servidor domótico

En esta unidad hemos visto el funcionamiento básico de Arduino, pero este no será el único dispositivo capaz de domotizar una casa. El servidor domótico será el encargado de gestionar todos los dispositivos que hemos montado con Arduino.

El servidor domótico podrá ser perfectamente un ordenador, pero implica tenerlo funcionando todo el tiempo y eso tampoco nos conviene. Por ello hay otros recursos como podría ser una placa **Raspberry Pi**.





 www.universae.com

